

Database Systems

Introduction



The Relational Data Model

✓ A brief history of database systems

- What is a data model?
- Important data models
- The relational data model
 - Basic Concepts
 - Constraints
 - Operations

Brief History of Database Systems

- Early important data applications
 - Banking systems
 - Airline reservation systems
 - Corporate record keeping
 - Supplies and parts
- Requirements of such applications
 - Small items of data
 - Many queries or modifications on data

Brief History of Database Systems

1959

- **CODASYL**(Committee on Data Systems Languages) introduces **COBOL** for business data processing
 - COmmon Business Oriented Language

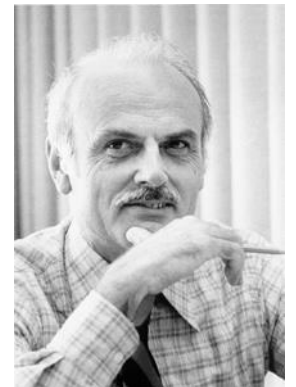
1960s

- Integrated Data Store: First general purpose DBMS by Charles Bachman at General Electric in early 1960s. Basis for **network data model**
- Information Management System (IMS): Developed by IBM

Brief History of Database Systems

1970s

- Previous to this point data models are **navigational**
- Edgar Codd Mathematician at IBM San Jose Research Lab
Proposed relational data model
 - Store database in simple data structures
 - Decouple high-level structure from the low-level storage details
- Early implementation of relation DBMS
 - System R (IBM Research)
 - INGRES (UC Berkeley)
 - Oracle (Larry Ellison)



Brief History of Database Systems

1980s

- SQL developed as part of IBM system R project and standardized in late 1980s
 - IBM DB2 commercialized 1983
- Relational model gained widespread use
- Start of today's two famous open source DBMS



1999

- SQL adopted and Standardized
 - American National Standards Institute (**ANSI**)
 - International Organization for Standardization (**ISO**)



SQL still has a very big market

Brief History of Database Systems

2000s

- Internet boom
 - Larger data volume and faster updates that could not be handled by single machine
 - Read-only web to read-write web
 - Scalability, Availability, Flexibility
 - Unstructured & semi-structured data

Brief History of Database Systems

2000s

- OnLine Transaction Processing (OLTP) & OnLine Analytical Processing (OLAP)
 - **OLTP** (High-frequency updates & small queries)
 - **NoSQL** vs. **Relational** DBMS?
 - **OLAP** (Low frequency updates & big queries)
 - **MapReduce** vs. **Relational** DBMS?

Brief History of Database Systems

2000s

- **NoSQL**

- Non-Relational data models (document, key-value)
- Non-**ACID** Transactions
- Custom APIs instead of SQL
- Usually open source



ACID Properties

A: Atomicity (all or nothing)

C: Consistency

I: Isolation

D: Durability

Brief History of Database Systems

2000s

- **Not Only SQL (NoSql)**
 - **Document** stores (Data Model: **JSON**) - Example Systems: SimpleDB, CouchBase, MongoDB
 - **Column** stores (Data Model: **Big Table**) - Example Systems: Hbase, Cassandra, HyperTable
 - **Key-value** stores (Data Model: **Hash**) - Example Systems: DynamoDB, Riak, Redis, Membase
 - **Graph data** stores (Data Model: **Graph**) - Example Systems: Neo4J, InfoGrid, GraphBase
- **NoSQL Limitations**
 - Low-level Language: Simple read & write database operators
 - Weak Consistency (Last, Stale): Eventual Consistency (All updated)
 - Lack of Standardization: 100+ NoSQL systems

Brief History of Database Systems

2000s

- MapReduce
 - Advantages
 - Fault tolerance
 - Complex analytics (SQL, ML, Graph Processing)
 - Heterogeneous storage systems
 - No data loading requirement
 - Limitations
 - Debate on
 - Missing features
 - Incompatibility with DBMS tools
 - Novelty
 - Quality of implementation

APACHE
HBASE



MapReduce a Program model rather than a database system

Brief History of Database Systems

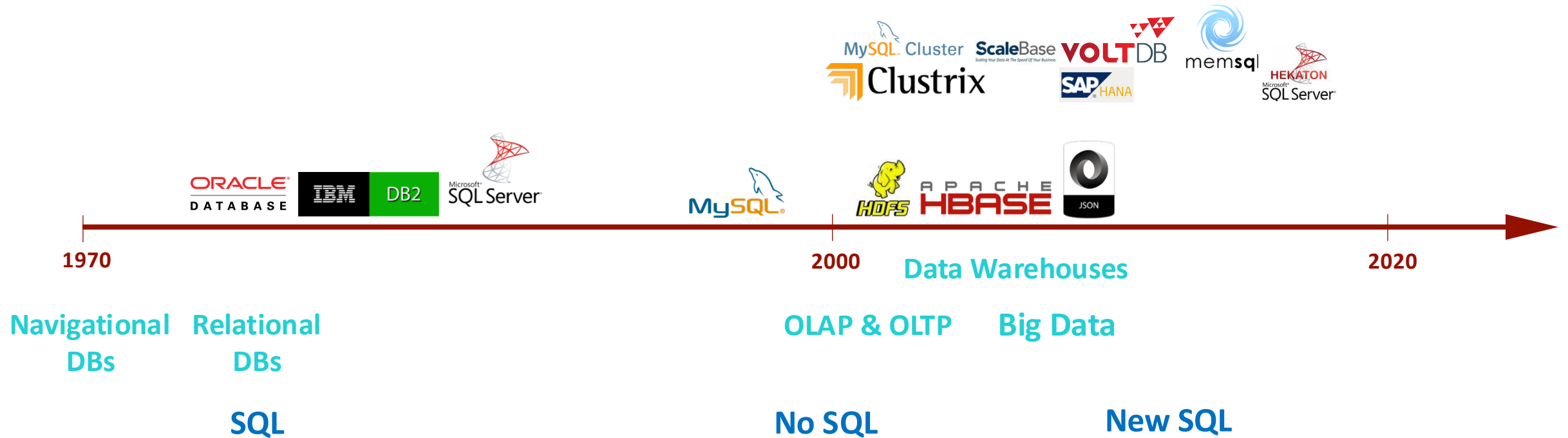
2010s

- NewSQL

- Provide same performance of OLTP workloads as NoSQL DBMS without giving up ACID
 - Relational / SQL
 - Distributed
 - Usually closed source
- Strong Consistency
- Features & limitations on flexibility, scalability and availability
- Systems: ScaleBase, MySQL Cluster, Hekaton SQL Server, Clustrix, SAP HANA, Pivotal



Database Systems



The Relational Data Model

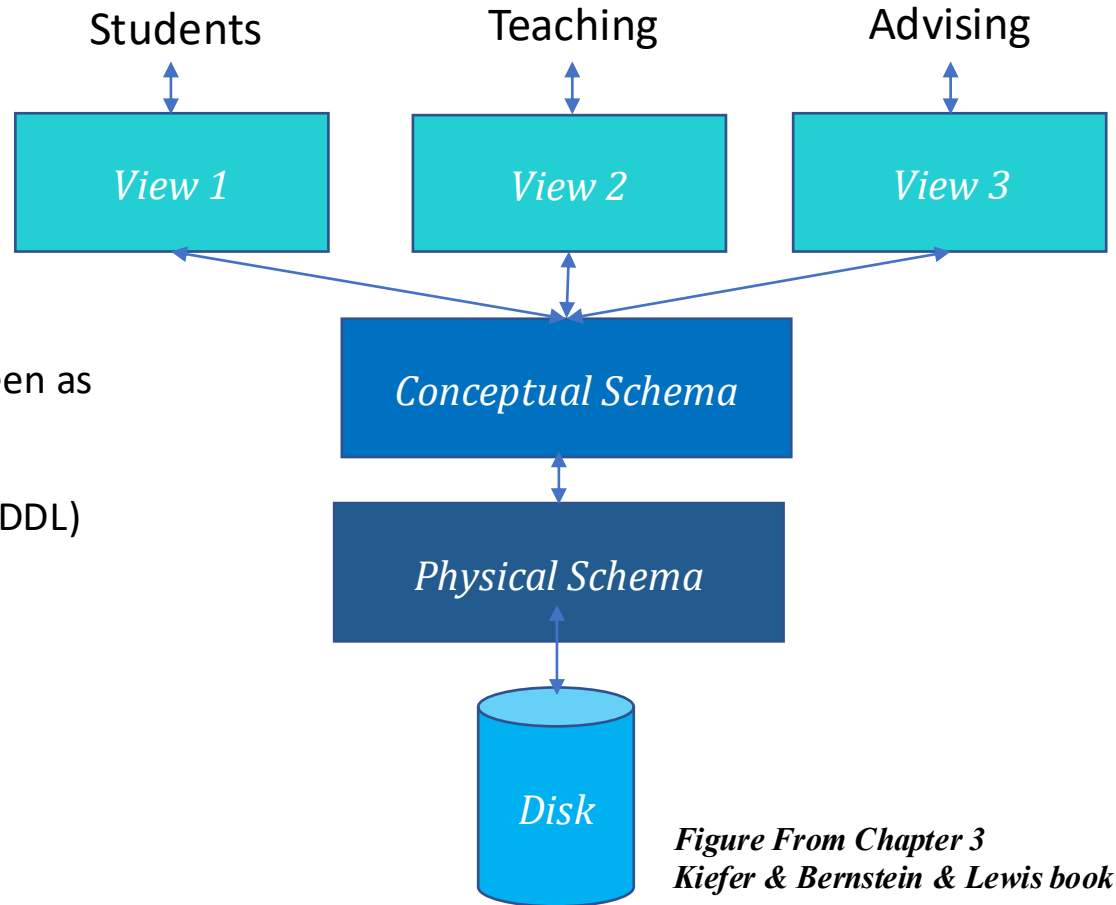
- A brief History of Database Systems

✓ What is a data model?

- Important data models
- The relational data model
 - Basic Concepts
 - Constraints
 - Operations

What is a Data Model?

- Notation for describing data or information
 - Consisting of
 - **Structure of data**
 - Schema: **Conceptual**
different from data structure that can be seen as
Physical data model
 - Defined using a **Data Definition Language (DDL)**
 - Operations on the data
 - Constraints on the data



*Figure From Chapter 3
Kiefer & Bernstein & Lewis book*

What is a Data Model?

- Notation for describing data or information
 - Consisting of
 - Structure of data
 - **Operations** on data
 - **Queries**: Operations that retrieve information
 - **Modifications**: Operations that change the database
 - Described by **Data Manipulation Language (DML)**
 - Different from programming languages
 - Support a set of limited operations
 - Allow for query optimizations
 - Constraints on data

What is a Data Model?

- Notation for describing data or information
 - Consisting of
 - Structure of data
 - Operations on data
 - **Constraints** on data
 - Specify conditions and limitations on the data, and ensure correctness of data: Integrity Constraints
 - Defined using DDL

The Relational Data Model

- A brief history of database systems
- What is a data model?

✓ Important data models

- The relational data model
 - Basic Concepts
 - Constraints
 - Operations

Data Models

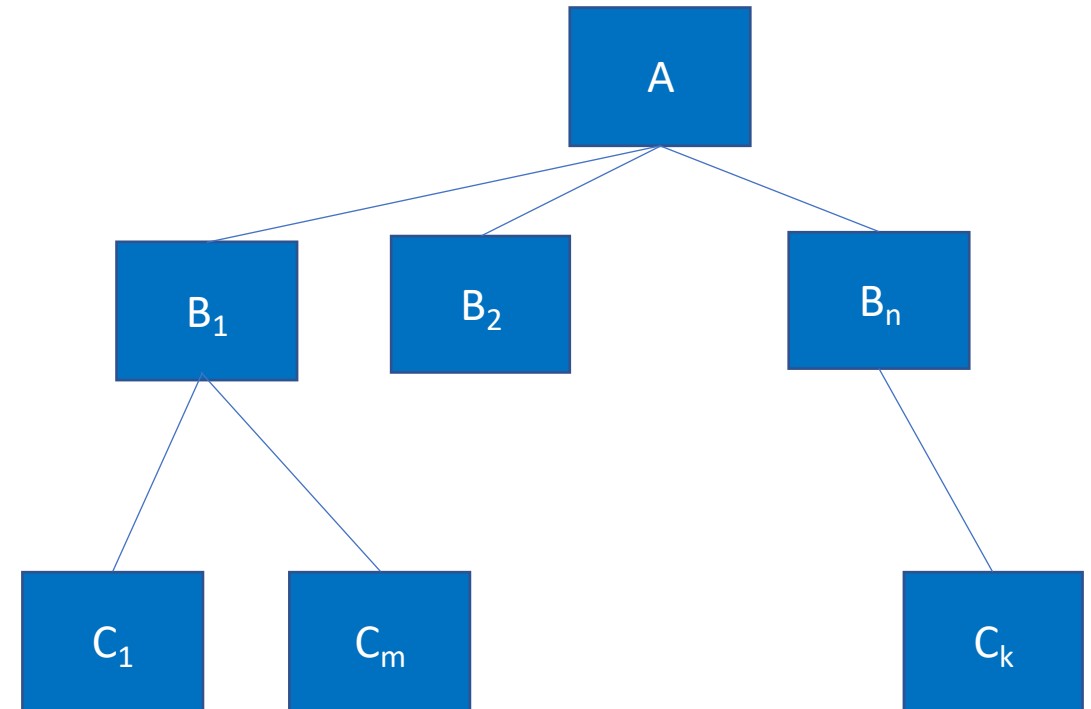
- **Data Model:** Collection of high-level data description constructs that hide many low-level storage details
 - Hierarchical Model (Tree)
 - Network Model (Graph)
 - Object-oriented Model
 - **Relational Data Model (Table)**
 - Key-value Data Model
 - Semi-structured Data Model (Tree)

Data

- Relational ← Most DBMSs: What is covered in **This Course!**
 - Key/Value
 - Graph
 - Document
 - Column-Family
 - Array/Matrix ← Machine Learning
 - Hierarchical
 - Network
- NoSQL
- Obsolete - Rare
-
- The diagram illustrates the categorization of various database types. A list of database types is shown on the left. A horizontal arrow points from 'Relational' to the text 'Most DBMSs: What is covered in This Course!'. A large curly brace groups 'Key/Value', 'Graph', 'Document', and 'Column-Family', with the label 'NoSQL' to its right. Another horizontal arrow points from 'Array/Matrix' to the text 'Machine Learning'. A smaller curly brace groups 'Hierarchical' and 'Network', with the label 'Obsolete - Rare' to its right.

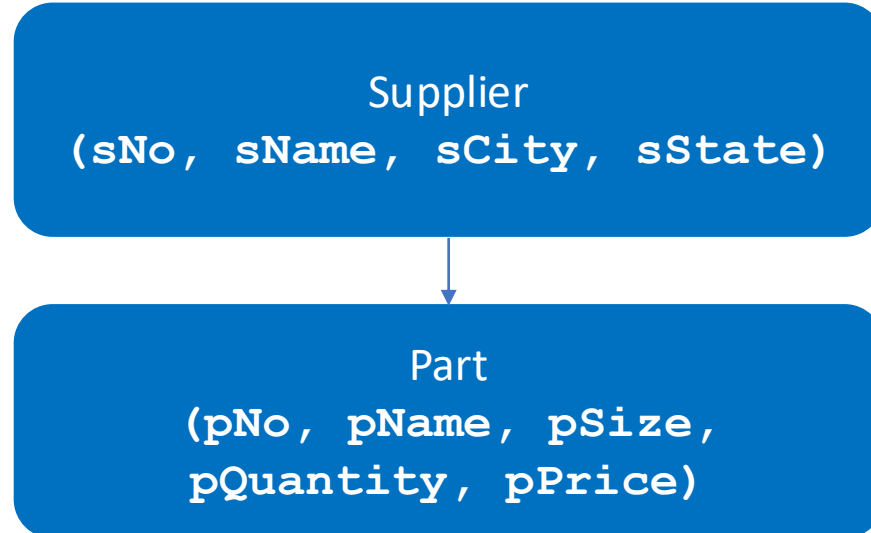
Hierarchical Model

- Tree-based structure
 - **Records:** Structure that holds data
 - **Links:** Relation between the records
- IBM Information Management System (IMS)
- Early 1960s



Hierarchical Model

- Example



sNo	sName	sCity	sState	parts
1001	First Supplier	New York	NY	
1002	Second Supplier	SF	CA	

pNo	pName	pSize	pQuantity	pPrice
999	Batteries	Large	61	\$100

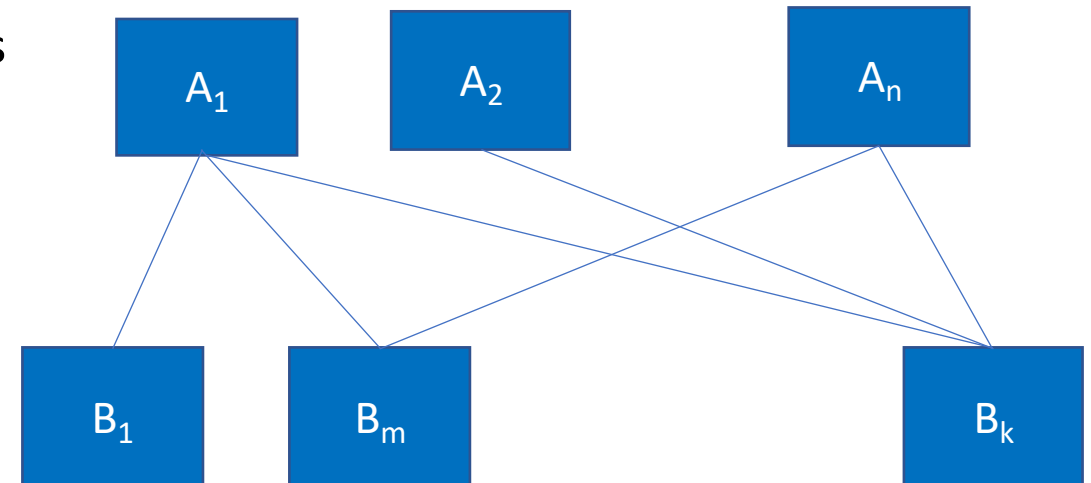
pNo	pName	pSize	pQuantity	pPrice
999	Batteries	Large	14	\$99

Diagram illustrating the hierarchical model structure and data:

- Supplier Table:** Contains two suppliers (1001, 1002) with their names, cities, and states. The 'parts' column is linked to the Part tables.
- Part Table (Supplier 1001):** Contains one part (999, Batteries, Large, 61, \$100).
- Part Table (Supplier 1002):** Contains one part (999, Batteries, Large, 14, \$99).

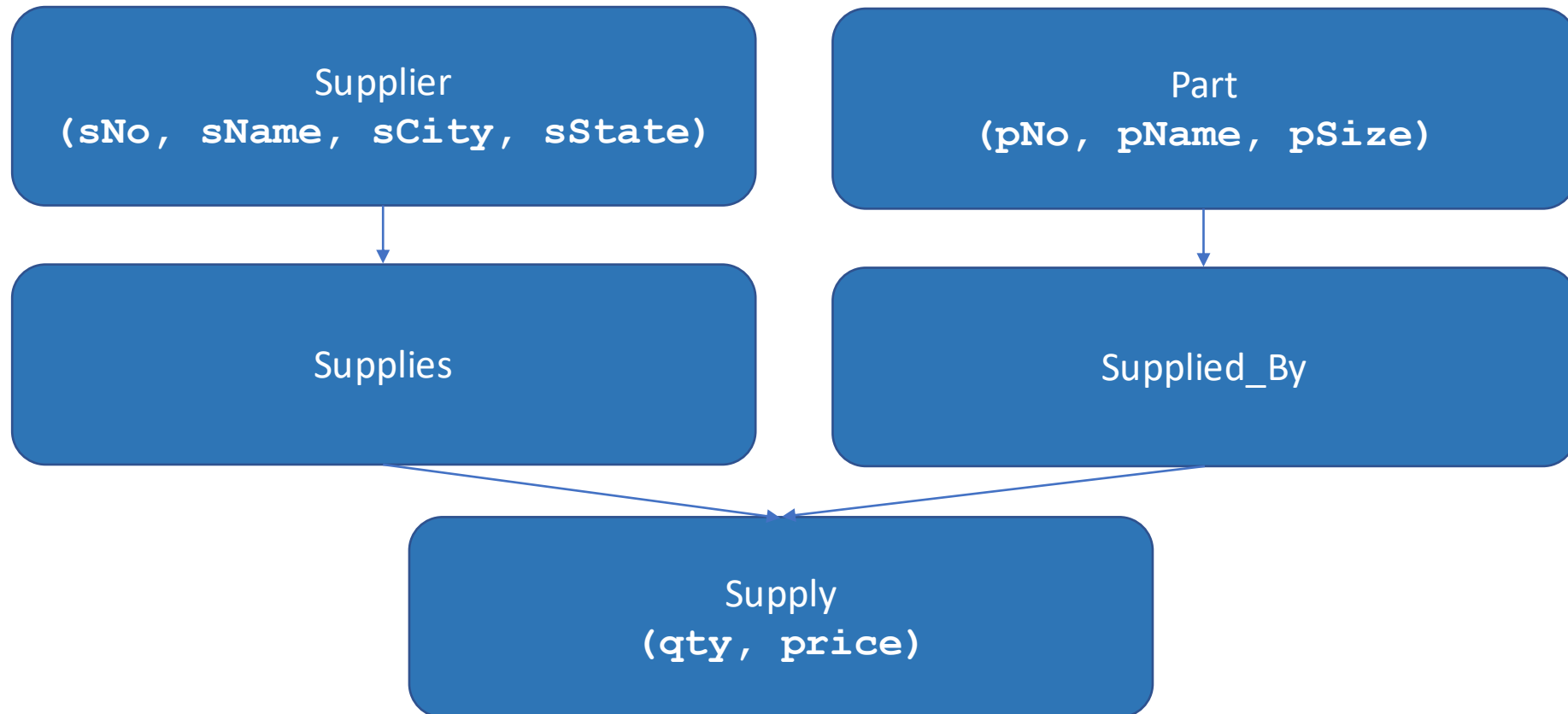
Network Model

- Graph-based structure
 - **Records:** Structure that holds data
 - **Sets:** One to many relations between the records



Network Model

- Example



Object-Oriented Model

- Complex data types
 - Non-atomic domains
 - Objects
- 1970s – now
- Few of the original DBMSs in this model exist today, but many of the technologies exist in other forms (XML, JSON)

Object-Oriented Model

- Example

Application Code:

```
class Student
{
    int id;
    String name;
    String email;
    String phone[];
}
```

id	name	email
1001	Adam Smith	adam@school.edu

sid	phone
1001	333.333.3333
1001	444.444.4444

Key-Value Data Model

- Structure of the Data
 - (Key, Value) pairs
 - Domain
 - Key is an integer or string
 - Value can be any object
- Constraints on the data
 - Example
 - Key is unique
 - Value is not NULL
- Query language
 - get(key)
 - put(key, value)

Key → Value
1000 → (Mike, 21, 3.8)
1001 → (Bill, 19, 3.4)
1002 → (Alice, 20, 3.6)

Semi-Structured Data Model

- Semi-structured-data Model
 - Structures: Trees
 - Example
 - XML.
 - Query Language: Xpath
 - Constraints
 - Example
 - <Length> has to be integer
 - Each <Movie> has a <Year> element nested within in

```
<Movies>
  <Movie title="Gone With the Wind">
    <Year>1939</Year>
    <Length>231</Length>
    <Genre>drama</Genre>
  </Movie>
  <Movie title="Star Wars">
    <Year>1977</Year>
    <Length>124</Length>
    <Genre>sciFi</Genre>
  </Movie>
  <Movie title="Wayne's World">
    <Year>1992</Year>
    <Length>95</Length>
    <Genre>comedy</Genre>
  </Movie>
</Movies>
```

From Chapter 2, The complete book

The Relational Data Model

- A brief history of database systems
- What is a data model?
- Important data models
- ✓ **The relational data model**
 - Basic Concepts
 - Constraints
 - Operations

Preference of Relational Models

- Relational Model provides
 - **Simple, limited** approach to structuring data
 - Yet reasonably versatile, so anything can be modeled
 - A limited collection of operations on data
 - Yet useful collection of operations
- Requirements of large databases
 - Efficiency of access
 - Ease of use: Development performance
- Tables **DO NOT** prescribe how they are implemented or stored on disk
 - This is called physical data independence

Relational Data Model

- Single way to represent data:
 - Two-dimensional **table** called a ***relation** or **relation instance***
- Relation instances are dynamic
 - We can add, delete, modify tuples

title	year	length	genre
Gone with the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

The relation Movies
From Chapter 2, The complete book

- **Attributes:** Columns of a relation named by attributes
- **Tuples:** Rows of a relation other than header (one component)
 - Components of each tuple are atomic. I.e. string, integer, etc., not an structure

Relational Data Model

- **Schema**: Name of a relation and set of attributes for a relation
Schema for relation Movies:

Movies(title, year, length, genre)

- **Relation schema**

- The name of a relation: **Movies**
- The set of attributes for a relation

(title, year, length, genre)

title	year	length	genre
Gone with the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

The relation Movies
From Chapter 2, The complete book

- Schema changes possible but expensive
 - Millions of tuples to be rewritten to add or delete components

Relational Data Model

- The set of schemas for the relations of a database is called a *relational database schema*, or just a *database schema*

```
Students(studentId, studentName, age, gpa)  
Course(courseId, courseName, credit)  
Take(studentId, courseId)
```

or

```
Movies(title, year, length, genre)  
Books(title, year, pages, genre)  
Music(title, year, length, genre)
```

Relational Data Model

- Tables: Relations
- Columns: Attributes, Fields
- Rows: Tuples, Records
- **Degree (arity)** of a relation = number of attributes
- **Cardinality** of a relation = number of tuples

The diagram shows a table with four columns: **id**, **name**, **age**, and **gpa**. The table contains three rows of data. A blue box labeled "Columns/ Attributes/ Fields" has arrows pointing to each of the four column headers. Another blue box labeled "Rows/ Tuples/ Records" has arrows pointing to each of the three data rows.

id	name	age	gpa
1000	Mike	21	3.8
1001	Bill	19	3.4
1002	Alice	20	3.6

Relational Data Model

- Relations are set of tuples (not a list of tuples)
 - Order does not matter (for rows)
 - Order is important in columns (change in schema, change in tuples)

Equivalent Representations of a Relation:

id	name	age	gpa
1000	Mike	21	3.8
1001	Bill	19	3.4
1002	Alice	20	3.6

=

id	name	gpa	age
1000	Mike	3.8	21
1001	Bill	3.4	19
1002	Alice	3.6	20

=

id	name	age	gpa
1000	Mike	21	3.8
1002	Alice	20	3.6
1001	Bill	19	3.4

How many different ways to represent this relation? **3! x 4! = 144**

Relational Data Model

- **Domain:** Associated with each attribute of a relation is a domain. The value of each components should be within the corresponding domain.
- Requirement: Data atomicity of the values in the domain
 - I.e. The relational model does not specify any means for looking into the internal structure of the value, so that the values appear indivisible to the relational operators
- Possible to include the domain or data type for each attribute in a schema:

`Movies(title:string, year:integer, length:integer, genre:string)`

`Students(studentId:integer, studentName:string, age:integer, gpa:real)`

Relational Data Model

- Domain Examples
 - Text: CHAR(20), VARCHAR(50), TEXT
 - Integer: INT, SMALLINT
 - Real: DOUBLE, FLOAT
 - Few more that are vendor specific

`Student(studentId:INT, studentName:VARCHAR(50), age:INT, gpa:FLOAT)`

Key

- An attribute or a set of attributes forms a key for a relation if we do not allow two tuples in a relation instance to have the same values in all the attributes of the key. The key is used to uniquely identify a record.

Example: Title and year are the key in movies relation

Accounts(acctNo, type, balance)

acctNo	type	balance
12345	savings	12000
23456	checking	1000
34567	savings	25

- Many real-world databases use artificial keys as may not be safe to make assumptions about the values of attributes
 - E.g. Employee ID

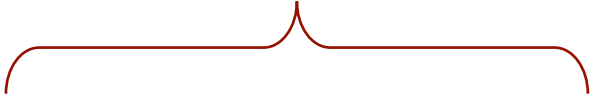
Multiple Keys

- A set of attributes form a key
- Example: Title and year are the key in movies relation

Movies (title, year, length, genre)

- Neither of these attributes are the key alone

Key= title + year



Title	Year	Length	Genre
Gone with the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

Foreign Key

- Attribute(s) whose value is a key of a record in some other relation
 - The attributes used to cross-reference relations need not have the same name
 - A foreign key can consist of more than one attribute

Accounts

acctNo	type	balance
12345	savings	12000
23456	checking	1000
34567	savings	25

Customers

fname	lname	idNo	account
Robbie	Banks	901-222	12345
Lena	Hand	805-333	12345
Lena	Hand	805-333	23456



Foreign key to Accounts: acctNo

Integrity Constraints (IC)

- An instance of a schema is legal if it satisfies all integrity constraints associated with the schema
 - Type constraint
 - Key constraints
 - Referential integrity constraints
 - Semantics constraints

Type Constraint

- To meet the type constraint, a relation instance must satisfy
 - Column naming
 - Matching between column name and associated record attribute
 - Domain constraints
 - The values that appear in each column must belong to the defined domain

Illegal

Title	Year	Length	Genre
Gone with the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	comedy	95	1992

Illegal

Title	Year	Length	Genre
Gone with the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95.5	comedy

Key Constraints

- If a relation has several keys, they are referred to as **candidate keys**. One key is usually used as **primary key**
- **Key constraint**
 - A relation instance of a schema should not contain a pair of distinct tuples whose values agree on all of the attributes in the key

Illegal

acctNo	type	balance
12345	savings	12000
23456	checking	1000
23456	savings	25

Referential Integrity

- The requirement that referenced tuples must exist is referential integrity
- Foreign key constraints
 - A key is a foreign key for a relation if and only if for every tuple there is a tuple that has the same values over the attributes in the referenced relation as does over the corresponding attributes in the original relation.

Accounts

acctNo	type	balance
12345	savings	12000
23456	checking	1000
34567	savings	25

Illegal

Customers

fname	lname	idNo	account
Robbie	Banks	901-222	12335
Lena	Hand	805-333	12355
Lena	Hand	805-333	23456

Foreign key to Accounts: acctNo

Semantic Constraints

- Implement a business rule or convention
- Little to do with structure
- Derived from application domain
- Example
 - Students should not register in more than 5 courses
 - Students should have prerequisites to register for a course
 - No employee should have more than one boss

Enforcing Integrity Constraints

- When integrity violated
 - Transaction aborted
 - Remedial action taken: Reactive constraints
- **Triggers (event-condition-action)**
 - Triggered by an event tests a *condition*
 - Condition does not hold then nothing happens in response to this event
 - Condition is satisfied, the *action* is performed by the DBMS
 - Action could be
 - Modify the effects of the event in some way (e.g. remedial action)
 - Aborting the transaction
 - The action could be any sequence of database operations, including operations not connected in any way to the triggering event

SQL

- **SQL: Structured Query Language**

- Data Definition Language (DDL)

The **Data-Definition** sublanguage for declaring database schemas

- Data Manipulation Language (DML)

The **Data-Manipulation** sublanguage for querying databases and for modifying the database

- SQL makes a distinction between

- **Stored relations (Tables)**: Exists in DB and can be modified and queried
 - **Views** (Defined by Computation): Not stored but constructed when needed
 - **Temporary tables**: Constructed by SQL processor when it performs its job of executing queries and data modifications

Acknowledgements

[1] Database Systems: The Complete Book, 2nd Edition
Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom
Prentice Hall, 2009

[2] Database System Concepts, Seventh Edition
Avi Silberschatz, Henry F. Korth, S. Sudarshan
McGraw-Hill, March 2019
www.db-book.com

[3] CMPT 354 (Jiannan Wang - SFU)
<https://sfu-db.github.io/cmpt354/>

Additional references and resources used in preparation of this course are listed on
<https://ouldoozbk.github.io/teaching/summer2026/DBS/>
or mentioned on each slide.